Universidad Tecnológica de Bolívar
Programa de Ingeniería Biomédica

Diseño Inicial de un Agente Inteligente (versión ajustada con desarrollo y pruebas)

CampaignTarget-Agent: agente de aprendizaje para priorizar clientes en campañas de marketing directo

Presentado por:
Jose David Quiñonez Rehenals — T00083600
Gabriel Antonio Gonzalez Puello — T00061463

Grupo PA 8 · Curso: Inteligencia Artificial · Segundo Parcial [Agentes Inteligentes]
Docente: Edwin Alexander Puertas Del Castillo
Cartagena de Indias, 19 de octubre de 2026

Tabla de cambios respecto a la entrega anterior

Cada fila indica la sección modificada, lo que decía, lo que dice ahora y la razón. Las cifras y afirmaciones se verificaron ejecutando de nuevo el código desde cero (Python 3.12, scikit-learn 1.9.1).

Sección	Qué decía	Qué dice ahora	Por qué cambió
Portada	Plantilla sin completar ([NOMBRE DE LA INSTITUCIÓN], [Nombre completo del estudiante]…).	Datos reales de la institución, el programa, los integrantes y el docente.	El documento no podía entregarse con campos de plantilla.
Fase 1 (nueva sección 0)	Matriz con solo CampaignTarget puntuado (4,75); el Agente B quedaba vacío y la justificación no podía reconstruirse.	Matriz completa con los dos agentes asignados (CampaignTarget y Symptom Triage), evidencia en cada celda, sensibilidad calculada para ambos y justificación.	La actividad exige que otra persona llegue al mismo agente aplicando los mismos pesos; con una columna vacía no era verificable.
§2 Tipo de agente	El crítico se describía, pero el código no tenía ningún módulo crítico.	Se implementó `criticar()` (Brier y sesgo de la probabilidad frente a la respuesta real) y el texto lo referencia.	Coherencia diseño–código: un agente de aprendizaje debe tener crítico explícito.
§6 Salidas y actuadores	Se prometía un CSV con la lista priorizada, pero el script solo escribía el log.	`actuar()` escribe `listas/lista_ciclo_N.csv` con la probabilidad de cada cliente.	La salida existía solo en el notebook, no en el repositorio reproducible.
§7 Medidas de desempeño	Cinco medidas; el registro completo se medía como «4 eventos por ciclo»; sin criterio para entradas inválidas.	El registro pasa a 5 eventos por ciclo (se añade `critica`) y se verifica la secuencia y el orden de las marcas de tiempo; se añade la medida M6 (robustez); se declaran los escenarios donde aplica cada umbral.	Al añadir el crítico cambia el conteo de eventos; la actividad exige casos de falla esperada y aplicabilidad de cada umbral definida antes de probar.
§7 Valores preliminares	Backtest con scikit-learn de Colab: conversión 36,4 %, mejora +212 %, captura 62 %.	Reejecutado con scikit-learn 1.9.1: conversión 36,7 %, mejora +213 %, captura 63 %; AUC-ROC 0,803 (igual).	Los números se verificaron en un entorno limpio; la diferencia de 0–1 punto se debe a la versión de la librería.
§8 Percepción y manejo de entradas	Decía que `percibir()` preparaba los datos y que la validación era limitada.	La preparación (excluir `duration`, bandera `nunca_contactado`, one-hot) la hace `preparar()`; `percibir()` valida el esquema y rechaza lotes vacíos, con columnas faltantes, tipos inválidos o más de 20 % de datos faltantes.	El texto atribuía al método una responsabilidad que estaba en otra función; la validación se hizo real y probable.
§8 Memoria	El registro JSONL tenía 4 eventos por ciclo.	Tiene 5 eventos por ciclo aceptado (y 2 — percepción y rechazo— en un ciclo rechazado).	Consecuencia del crítico y de la validación.
§9 (nueva) Desarrollo del agente	No existía.	Arquitectura, tecnologías, entradas, salidas, diagramas de secuencia, diferencias, pruebas, limitaciones y enlaces.	Requisito de la Fase 2 de la actividad.
§10 Conclusiones	Conclusiones basadas en el backtest preliminar (sección 9 del documento anterior).	Pasa a ser la sección 10 y se actualiza con los resultados de las pruebas de la Fase 3.	La actividad inserta el desarrollo antes de las conclusiones.
§7 / Fase 3: reglas de las pruebas	En la corrida inicial en Colab (60 corridas) los umbrales M1–M5 se aplicaron a todos los escenarios y M5 esperaba 4 eventos por ciclo; 7 incumplimientos: M3 en «10 %», M3 y M4 en «1 %», M1 en «100 %», M4 en «90 % de fallos», M5 en «sin columna» y «lote vacío».	Los umbrales no cambian. Se declara en qué escenarios aplica cada medida (p. ej. M1 y M3 no tienen sentido al contactar al 100 % o al 1 %), M5 pasa a 5 eventos, se añade M6 y el agente valida entradas. Se reportan también los resultados con la regla original (9.7).	La corrida inicial mostró que el agente no validaba entradas (M5 = 85 % en lote vacío o sin columna) y que algunos umbrales no aplican a todos los escenarios. Esta reformulación se hizo DESPUÉS de ver la corrida inicial, por eso se publican ambas lecturas.
Retroalimentación docente (Tarea 2): documento incompleto	Solo se recibió el anexo técnico del dataset Bank Marketing; faltaban portada, planteamiento del problema, tipo de agente, tipos de ambiente, entradas/salidas y los 7 componentes IBM, y el anexo remitía a un «documento base» no entregado.	El documento es ahora autocontenido: portada, 1 planteamiento, 2 tipo de agente, 3 ambientes, 4 entorno, 5 entradas y sensores, 6 salidas y actuadores, 7 medidas SMART, 8 los siete componentes IBM, y el anexo del dataset incorporado como Anexo A.	Atender la observación del docente: el documento base y el anexo viajan juntos en un único PDF.

Fase 1 — Selección justificada del agente

El equipo tenía asignados dos agentes: **CampaignTarget-Agent** (agente de aprendizaje que prioriza clientes de una campaña bancaria) y **Symptom Triage Agent** (agente conversacional de pre-triage clínico, basado en metas; documento «Diseño Inicial de un Agente Inteligente» de la Tarea 2). Se compararon con una matriz ponderada; los criterios y pesos se definieron antes de puntuar y suman 100 %.

Paso 1. Criterios y pesos

Criterio	Qué evalúa	Cómo se mide	Peso
Viabilidad técnica	Si pueden construirlo con sus conocimientos y el tiempo disponible	Componentes que ya saben implementar / total	25 %
Disponibilidad de datos o entorno	Si existe o pueden simular el entorno, los sensores y los datos	Fuente identificada, simulador o dataset concreto	20 %
Medibilidad del desempeño	Si sus medidas se pueden probar de forma objetiva	Medidas con fórmula y umbral / total de medidas	20 %
Complejidad de IA	Si exige razonamiento real, no solo condicionales	Tipo de agente según Russell y Norvig	15 %
Costo y recursos	Dinero, hardware, APIs pagas o cuotas	Costo estimado en COP o en llamadas de API	10 %
Valor del problema	Utilidad real para un usuario identificable	Usuario y beneficio concretos	10 %
Total			**100 %**

Pasos 2 y 3. Puntajes con evidencia y puntaje ponderado

Criterio	Peso (w)	CampaignTarget: puntaje	CampaignTarget: evidencia	Symptom Triage: puntaje	Symptom Triage: evidencia
Viabilidad técnica	0,25	5	Los 6 componentes del agente (percepción, decisión, acción, crítico, aprendizaje y simulador) están implementados y probados con pandas y scikit-learn, herramientas que el equipo ya domina: 6/6.	3	De los 7 componentes IBM del diseño, el equipo sabe implementar unos 4 (memoria de conversación, reglas de alarma, clasificador y agenda simulada); el NLU clínico en lenguaje natural y la validación con profesionales exceden el tiempo disponible: ≈4/7.
Disponibilidad de datos o entorno	0,20	5	Bank Marketing (UCI): 45.211 registros públicos con 16 variables y objetivo, más un simulador propio (SimuladorBanco) que entrega percepciones y respuestas.	2	El diseño no identifica un dataset conversacional de triage; habría que construir los 150 casos simulados y etiquetarlos con profesionales de salud, a quienes el equipo no tiene acceso.
Medibilidad del desempeño	0,20	5	Las 6 medidas (M1–M6) tienen fórmula y umbral numérico: 6/6, probables con el log del agente.	3	Una sola medida con fórmula y umbral (concordancia ≥ 80 % en 150 casos): objetiva, pero depende de etiquetas de un profesional que no están disponibles.
Complejidad de IA	0,15	4	Agente de aprendizaje (gradient boosting con reentrenamiento por ciclo). Sin planificación dinámica ni generador de problemas: no llega a 5.	4	Agente basado en metas con estado interno, selección dinámica de preguntas y reglas de seguridad para escalar: mayor razonamiento que un modelo de ranking.
Costo y recursos	0,10	5	Cómputo local o Colab gratuito; entrena en ~2 s; $0 COP y sin llamadas a APIs de pago.	3	Requeriría NLU o un modelo de lenguaje (API de pago o modelo local) sin costo estimado en el diseño; sin hardware especial.
Valor del problema	0,10	4	Usuario identificable (equipo comercial de un banco) y beneficio medible (+213 % de conversión en backtest), pero los datos son de 2008–2010 de una sola entidad.	5	Usuario claro (personal de triage en urgencias saturadas) y beneficio de alto impacto clínico, aunque con supervisión humana obligatoria.
Puntaje ponderado S	100 %	**4,75**	$S = \Sigma\, w_i \cdot p_i$	**3,15**	$S = \Sigma\, w_i \cdot p_i$

S(CampaignTarget) = (0,25 × 5) + (0,20 × 5) + (0,20 × 5) + (0,15 × 4) + (0,10 × 5) + (0,10 × 4) = **4,75**.
S(Symptom Triage) = (0,25 × 3) + (0,20 × 2) + (0,20 × 3) + (0,15 × 4) + (0,10 × 3) + (0,10 × 5) = **3,15**.

Paso 4. Análisis de sensibilidad

El criterio que más separa a los dos agentes en la suma ponderada es **Disponibilidad de datos o entorno** (w · diferencia de puntaje = 0,60). Se cambió su peso en ±10 puntos y el resto de pesos se redistribuyó proporcionalmente para que Σ w = 1. Se repitió con la viabilidad técnica, el criterio de mayor peso.

Criterio modificado	Escenario	Peso	S (CampaignTarget)	S (Symptom Triage)	¿Se mantiene CampaignTarget?
Disponibilidad de datos o entorno	Base	20,0%	4,75	3,15	Sí
Disponibilidad de datos o entorno	+10 puntos	30,0%	4,78	3,01	Sí
Disponibilidad de datos o entorno	−10 puntos	10,0%	4,72	3,29	Sí
Viabilidad técnica	Base	25,0%	4,75	3,15	Sí
Viabilidad técnica	+10 puntos	35,0%	4,78	3,13	Sí
Viabilidad técnica	−10 puntos	15,0%	4,72	3,17	Sí

Prueba adicional de robustez: si se mejorara la puntuación del Agente B a 5 en viabilidad técnica y en disponibilidad de datos (sus dos criterios más débiles), su puntaje sería 4,25, todavía inferior a 4,75. Con los puntajes más conservadores del primer borrador de la matriz (Medibilidad 4 y Complejidad de IA 3, el resto igual) CampaignTarget obtiene 4,40, aún por encima de 3,15. Para empatar a CampaignTarget, el criterio «Valor del problema», el único donde B gana, tendría que pesar al menos 65,4% (hoy pesa 10 %), con los demás pesos reescalados.

Paso 5. Justificación de la selección

Se elige **CampaignTarget-Agent** con S = 4,75 sobre 5, frente a 3,15 del Symptom Triage Agent. La decisión se reconstruye con la tabla: CampaignTarget gana en los tres criterios de mayor peso (viabilidad, datos y medibilidad) y también en costo, empata en complejidad de IA (4 y 4) y solo pierde en valor del problema (4 frente a 5). Sus puntos faltantes están en complejidad de IA (no planifica ni usa un generador de problemas) y en valor del problema (datos de 2008–2010 de una sola entidad). La sensibilidad no cambia el resultado: en todos los escenarios evaluados CampaignTarget queda entre 4,72 y 4,78 y Symptom Triage entre 3,01 y 3,29, sin cruces.

Qué se pierde al descartar el otro agente. Symptom Triage ofrece un problema de mayor impacto social (apoyar la priorización clínica en urgencias saturadas) y un agente más rico en razonamiento (planificación de preguntas, memoria conversacional, reglas de escalamiento). Esa ventaja no compensa su desventaja en los criterios de mayor peso: no hay datos ni etiquetas de profesionales para entrenar y probar la medida de concordancia ≥ 80 %, y construir un NLU clínico fiable excede el tiempo disponible. Además, un error en triage tiene un costo de seguridad del paciente que exigiría validación clínica fuera del alcance de esta actividad. Queda como trabajo futuro con un simulador de casos basado en las guías ESI.

1. Introducción y planteamiento del problema

Las campañas de marketing directo de una entidad financiera contactan a miles de clientes, pero solo una minoría responde de forma positiva. En el conjunto de datos público Bank Marketing (UCI), que recoge campañas de telemercadeo de un banco portugués, solo el 11,7 % de 45.211 contactos terminó en la suscripción de un depósito a plazo. Contactar a todos los clientes sin criterio implica costos de llamadas, tiempo del equipo comercial y desgaste de clientes que no estaban interesados.

Un agente inteligente es una solución pertinente porque la decisión de a quién contactar se repite en cada campaña, se apoya en datos del cliente y recibe retroalimentación verificable (quién respondió y quién no), de modo que puede mejorar con la experiencia. CampaignTarget-Agent estima la probabilidad de respuesta de cada cliente candidato, ordena la lista, entrega a la plataforma de campañas solo a los de mayor probabilidad, registra el resultado y reentrena su modelo. La decisión de cuántos clientes contactar (parámetro de campaña) permanece en manos del equipo comercial.

2. Tipo de agente inteligente

CampaignTarget-Agent es un **agente de aprendizaje**, según la clasificación de Russell y Norvig (AIMA). Cumple los componentes de esa arquitectura: el *elemento de desempeño* percibe el lote de clientes y decide a quiénes contactar (métodos `percibir()`, `decidir()` y `actuar()`); el *crítico* compara la probabilidad predicha con la respuesta real del cliente tras actuar (método `criticar()`, que registra el error de Brier y el sesgo de la probabilidad); y el *elemento de aprendizaje* añade esa experiencia al conjunto de entrenamiento y reentrena el modelo en cada ciclo (método `aprender()`).

No es un agente de reflejo simple porque no aplica reglas condición-acción fijas, sino un modelo estadístico (gradient boosting) entrenado con datos históricos y actualizado con nuevos resultados. Tiene además un componente de utilidad, ya que ordena a los clientes por probabilidad esperada de conversión, pero el rasgo que lo define es que su desempeño mejora con la experiencia. En su forma actual no incluye un *generador de problemas*, es decir, no explora de forma deliberada clientes de baja probabilidad para aprender de ellos; esa limitación se discute en las conclusiones.

3. Tipos de ambiente

Observabilidad: parcialmente observable. El agente conoce el perfil del cliente y su historial de contacto, pero no su intención de compra, su situación financiera actual ni sus motivos. Además hay vacíos en los datos: el resultado de la campaña previa es desconocido en cerca del 82 % de los registros y el canal de contacto en cerca del 29 %. Tampoco observa qué habrían respondido los clientes que decide no contactar.

Determinismo: estocástico. Dos clientes con el mismo perfil pueden responder distinto y el mismo cliente puede responder distinto en otra campaña. A esto se suma que los contactos pueden fallar (cliente no alcanzado); el simulador modela este efecto con una tasa de fallo del 10 %.

Episodicidad: secuencial. Dentro de un lote, la decisión sobre cada cliente es independiente, pero entre ciclos no lo es: los resultados de los clientes contactados se incorporan al entrenamiento y cambian las decisiones futuras.

Dinamismo: estático dentro de cada ciclo, con cambio lento entre campañas. La lista de candidatos no cambia mientras el agente decide, pero el comportamiento de los clientes y las condiciones económicas pueden variar de una campaña a otra (deriva de los datos), lo cual justifica el reentrenamiento periódico.

Discretización: discreto. Conjunto finito de clientes, tiempo por ciclos (lotes de campaña) y acción discreta sobre cada cliente: contactar o no contactar. Algunas variables de entrada son numéricas, pero percepciones y acciones ocurren en pasos discretos.

Número de agentes: agente único. No hay otros agentes que compitan o cooperen con CampaignTarget-Agent; los clientes responden a la oferta pero no interactúan estratégicamente con él, y el equipo comercial actúa como supervisor.

4. Descripción del entorno

Contexto. El agente opera en el área de marketing de una entidad financiera que realiza campañas de venta directa (en el caso de estudio, depósitos a plazo por telemercadeo).

Actores. Clientes candidatos; equipo comercial, que define la fracción de clientes a contactar y ejecuta las llamadas; equipo de datos, que mantiene el modelo; la plataforma de campañas o CRM, que recibe la lista y devuelve los resultados; y el área de cumplimiento y privacidad.

Restricciones del dominio. Capacidad limitada de llamadas por campaña; consentimiento del cliente y normas de protección de datos; variables sensibles (edad, estado civil) que requieren auditoría de equidad; y exclusión de la variable `duration` (duración de la llamada), porque solo se conoce después de contactar al cliente y usarla sería una fuga de información.

Condiciones normales. Lotes de miles de clientes con una tasa de respuesta cercana al 12 %, datos tabulares completos (valores desconocidos tratados como categoría) y respuestas conocidas poco después de cada campaña.

Condiciones excepcionales. Contactos fallidos, lotes vacíos o con columnas faltantes o con tipos inválidos, y cambios en la economía o en el perfil de los clientes que alteren la tasa de respuesta. En la implementación, el entorno real se sustituye por un simulador (`SimuladorBanco`) que entrega los clientes por lotes, revela la respuesta solo después de que el agente actúa e introduce fallos de contacto aleatorios; el agente rechaza y registra los lotes inválidos sin detenerse.

5. Entradas y sensores

Las percepciones del agente son los datos de cada cliente candidato y el resultado de cada contacto. La variable `duration` se excluye deliberadamente.

Entrada / percepción	Tipo de dato	Sensor o mecanismo
Edad, saldo anual (balance), día del mes, contactos en la campaña actual (campaign), días desde el último contacto (pdays), contactos previos (previous)	Numérico	Exportación de la base de clientes o CRM. En la implementación: `SimuladorBanco.llegan_clientes()` sobre bank-full.csv.
Ocupación, estado civil, educación, mora (default), préstamo de vivienda, préstamo personal, canal de contacto, mes, resultado de la campaña previa (poutcome)	Categórico	Misma exportación de la base de clientes.
Resultado del contacto (el cliente suscribió: sí / no)	Binario	Respuesta que devuelve la plataforma de campañas después de actuar. En la implementación: `SimuladorBanco.contactar()`.
Parámetros de campaña (fracción de clientes a contactar, número de lotes)	Numérico	Configuración definida por el equipo comercial (argumentos de línea de comandos o controles de la interfaz).

6. Salidas y actuadores

Las acciones del agente son la lista priorizada, el registro de ejecución y la actualización de su modelo.

Salida / acción	Actuador	Cómo modifica el entorno
Lista priorizada del 20 % de clientes con mayor probabilidad de respuesta, con su probabilidad	Envío a la plataforma de campañas. En la implementación: `actuar()` hacia `SimuladorBanco.contactar()` y un archivo `listas/lista_ciclo_N.csv`.	Los clientes de la lista reciben la oferta; los demás no se contactan. Genera las respuestas que el agente observa.
Registro de ejecución (log_ejecucion.jsonl) con percepciones, decisiones, acciones, crítica y aprendizaje, con marca de tiempo	Escritura de archivo	No cambia el entorno externo. Deja trazabilidad para auditoría y pruebas.
Modelo actualizado tras cada ciclo	Reentrenamiento (`aprender()`)	Cambia el criterio con que se decidirá en los ciclos siguientes.

7. Medidas de desempeño

Específica. Lograr que, al contactar al 20 % de clientes que el agente prioriza, la tasa de conversión sea al menos 20 % superior a la de una campaña sin priorización.

Medible. Conversión = suscripciones / clientes contactados y alcanzados. Mejora = conversión del agente / conversión base − 1, con umbral ≥ 20 %. La conversión base es la de contactar sin priorización (cerca de 11,7 % en los datos).

Alcanzable. Un backtest preliminar con gradient boosting (scikit-learn 1.9.1) dio AUC-ROC de 0,803 y una conversión de 36,7 % al contactar al 20 % superior (mejora de +213 %), muy por encima del umbral. Una simulación de 5 ciclos dio mejoras entre +203 % y +228 %.

Relevante. Contactar solo a los clientes más propensos reduce el costo de llamadas y el desgaste de clientes sin interés.

Con plazo. Se evalúa en las pruebas de la Fase 3 sobre la muestra de prueba del 20 % de los datos (9.043 clientes), con 5 repeticiones por escenario, antes de entregar el documento.

Medidas complementarias. M2: AUC-ROC ≥ 0,75 (calidad del ranking); M3: captura de compradores en el 20 % superior ≥ 50 %; M4: desviación estándar de la mejora entre repeticiones ≤ 10 puntos; M5: registro completo (5 eventos por ciclo aceptado, 2 por ciclo rechazado, marcas de tiempo ordenadas) = 100 %; M6 (añadida en la Fase 3): robustez, es decir, 100 % de ciclos terminados de forma controlada y correcta ante entradas inválidas o entorno adverso. La aplicabilidad de cada umbral a cada escenario se fijó antes de ejecutar las pruebas (archivo `umbrales_fase3.json`).

8. Componentes de la arquitectura de IA agéntica (marco IBM)

Los agentes de inteligencia artificial son sistemas capaces de percibir su entorno, tomar decisiones y ejecutar acciones de manera autónoma. De acuerdo con la arquitectura general de la IA agéntica descrita por IBM, sus componentes principales se estructuran en siete áreas funcionales. Fuente: IBM, «Components of AI agents» — https://www.ibm.com/mx-es/think/topics/components-of-ai-agents. A continuación se describe cómo CampaignTarget-Agent implementa cada una.

Percepción y manejo de entradas. El agente recibe un lote de clientes (`percibir()`) ya preparado por `preparar()`, que excluye `duration`, convierte `pdays = -1` en la bandera `nunca_contactado`, trata los valores desconocidos como categoría propia y codifica las variables categóricas (42 columnas finales). `percibir()` valida el esquema: rechaza y registra los lotes vacíos, con columnas faltantes, con tipos no numéricos o con más de 20 % de datos faltantes en una columna.

Planificación y descomposición de tareas. El plan es fijo y no se descompone de forma dinámica: en cada ciclo el agente puntúa a los clientes, los ordena, recorta al 20 % superior, contacta, critica y aprende. El plan de campaña lo definen dos parámetros que fija el equipo comercial: la fracción de clientes a contactar y el número de lotes. No usa un planificador ni un LLM.

Memoria. Corto plazo: el lote de clientes del ciclo actual. Largo plazo: el conjunto de entrenamiento acumulado (perfiles y respuestas de clientes contactados antes) y el modelo reentrenado. El registro de ejecución (JSONL) funciona como memoria de eventos, con percepciones, decisiones, acciones, críticas y aprendizajes fechados.

Razonamiento y toma de decisiones. Un modelo de gradient boosting (`HistGradientBoostingClassifier`, con pesos de clase balanceados) estima la probabilidad de que cada cliente suscriba. La decisión es una regla determinista: ordenar por probabilidad y contactar al 20 % superior. Se comparó con regresión logística (AUC-ROC de 0,772 frente a 0,803) y se eligió gradient boosting.

Llamada de herramientas y acción (tool calling). Las herramientas son funciones de Python: el modelo de scikit-learn, el procesamiento con pandas y la interfaz de contacto. En un despliegue real, esa interfaz sería la API de la plataforma de campañas o del CRM; en la implementación es `SimuladorBanco.contactar()`. La lista priorizada se exporta como CSV. No hay llamadas a un LLM.

Comunicación. El agente entrega al equipo comercial la lista priorizada con la probabilidad de cada cliente, el registro de ejecución y una tabla de resultados por ciclo (en consola y en la interfaz Streamlit). No tiene interfaz conversacional. Como mejora futura podría explicar por qué un cliente quedó alto en la lista.

Aprendizaje y adaptación. En cada ciclo (`aprender()`) añade los clientes alcanzados, con su respuesta real, al conjunto de entrenamiento y reentrena el modelo; el crítico mide cuánto se equivocó la predicción. Aprende solo de quienes contacta, lo que introduce un sesgo de retroalimentación. Además se prevé recalibrar el modelo periódicamente con campañas completas.

9. Desarrollo del agente

9.1 Arquitectura

El agente es un módulo de Python (`campaign_target_agent.py`) con tres piezas: `SimuladorBanco` (ambiente), `CampaignTargetAgent` (el agente, con un método por etapa del ciclo percepción → decisión → acción → crítica → aprendizaje) y funciones de datos (`preparar`, `dividir`, `modelos_base`). Dos interfaces lo invocan: la línea de comandos (`main()`) y la aplicación Streamlit (`app.py`); `pruebas_fase3.py` lo ejercita con 15 escenarios.

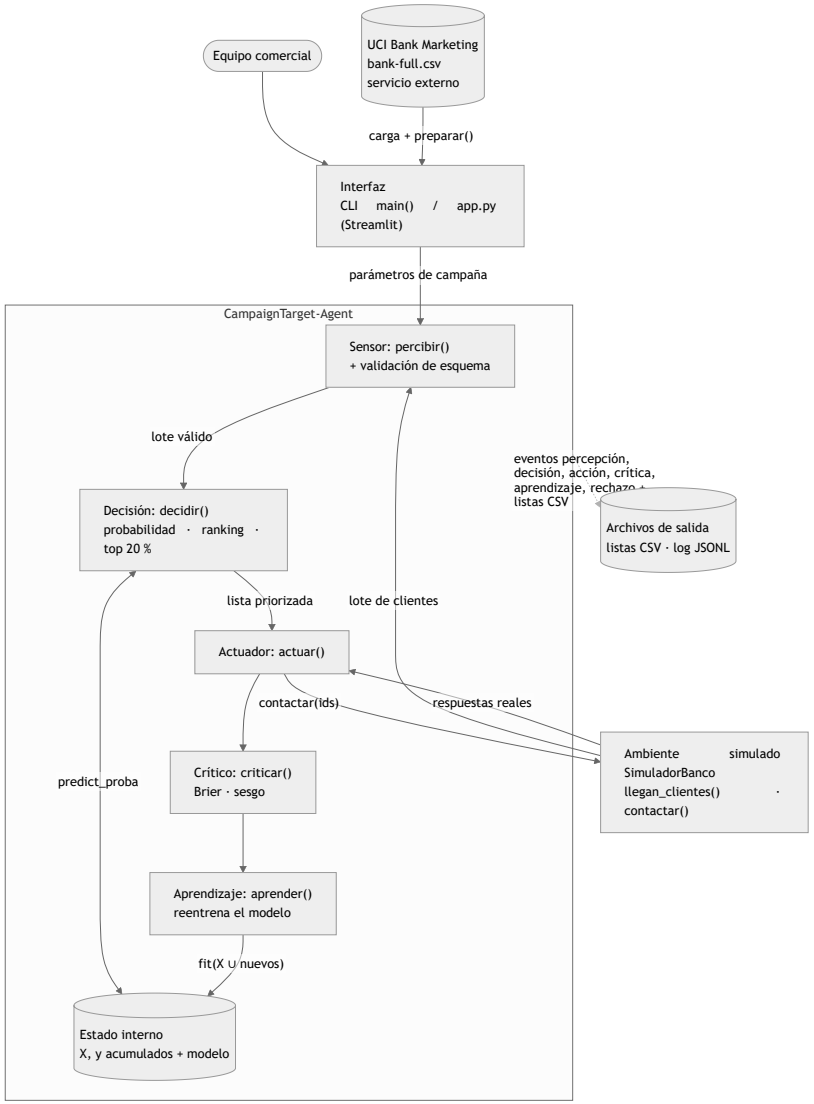


Figura 1. Diagrama de componentes: sensores, estado interno, módulo de decisión, actuadores y servicios externos. Las flechas punteadas son escrituras al registro y a los CSV.

Módulo del código que implementa cada componente IBM

Componente IBM	Módulo del código	Estado
Percepción y manejo de entradas	`preparar()` + `CampaignTargetAgent.percibir()` / `_validar()`	Implementado
Planificación y descomposición de tareas	`CampaignTargetAgent.ejecutar_ciclo()` y parámetros `frac`, `tandas`	Parcial: plan fijo; sin planificador dinámico (el problema no lo requiere: la decisión es un ranking)
Memoria	Atributos `X`, `y` y `modelo` del agente; `log_ejecucion.jsonl`	Implementado
Razonamiento y toma de decisiones	`decidir()` con `HistGradientBoostingClassifier`; `criticar()`	Implementado
Llamada de herramientas y acción	`actuar()` → `SimuladorBanco.contactar()` y CSV de la lista	Implementado contra un simulador (no hay CRM real)
Comunicación	Tabla por ciclo, log y `app.py` (Streamlit)	Parcial: sin interfaz conversacional ni explicación por cliente
Aprendizaje y adaptación	`aprender()`: reentrenamiento por ciclo	Implementado; sin generador de problemas (exploración)

9.2 Tecnologías

Qué razona el código y qué hacen las herramientas externas. Todo el razonamiento del agente lo hace el código propio: la validación, la regla de decisión (ordenar y recortar al 20 %), el crítico y el ciclo de aprendizaje. La herramienta externa scikit-learn solo ajusta y evalúa el modelo estadístico (`fit`, `predict_proba`). **No se usa ningún LLM** dentro del agente: no hay modelo de lenguaje, ni prompt de sistema, y ninguna decisión del agente depende de uno. Un asistente de IA (Claude Code) se utilizó solo como ayuda de desarrollo y documentación; ese uso se declara en la bitácora de uso de IA.

Tecnología	Versión	Para qué se usó	Por qué frente a una alternativa
Python	3.12.10	Lenguaje del agente, simulador y pruebas	Ecosistema de ciencia de datos; el equipo lo domina (frente a R o Java)
pandas	3.0.6	Carga, limpieza, one-hot y manejo de lotes	Estándar para datos tabulares; índices que permiten ligar cliente y respuesta
NumPy	2.5.4	Aleatoriedad del simulador (`default_rng`) y cálculos	Generador reproducible por semilla
scikit-learn	1.9.1	`HistGradientBoostingClassifier` (agente), regresión logística (comparación), métricas y partición	Gradient boosting nativo sin dependencias extra y con datos faltantes; AUC 0,803 frente a 0,772 de la logística. Frente a XGBoost/LightGBM: sin instalación adicional
Matplotlib	3.11.2	Curva de ganancia	Suficiente para un solo gráfico
Streamlit	1.66.0	Interfaz y despliegue web de la demo	Despliegue gratuito en Streamlit Cloud en pocas líneas; frente a Flask/Gradio requiere menos código
JSON Lines (JSONL)	—	Registro de ejecución con marca de tiempo	Un evento por línea, fácil de agregar y verificar; frente a SQLite o CSV no requiere esquema fijo
Mermaid	11.4.0	Diagramas UML del documento	Diagramas como texto, fáciles de corregir; frente a draw.io no requiere editor manual

9.3 Entradas (inputs)

Percepción	Tipo	Rango o formato	Origen	Ejemplo real
age	entero	18 – 95	archivo/CRM	47
balance	entero	−8.019 – 102.127 (euros)	archivo/CRM	1506
day	entero	1 – 31	archivo/CRM	5
campaign	entero	1 – 63	archivo/CRM	1
pdays → pdays + nunca_contactado	entero + bandera 0/1	pdays 0 – 871; −1 se convierte en bandera = 1	archivo/CRM	−1 → pdays = 0, nunca_contactado = 1
previous	entero	0 – 275	archivo/CRM	0

Percepción	Tipo	Rango o formato	Origen	Ejemplo real
job, marital, education, default, housing, loan, contact, month, poutcome	categórico (one-hot, 42 columnas finales)	job: 12 valores; marital: 3; education: 4; default/housing/loan: 2; contact: 3; month: 12; poutcome: 4 (incluye «unknown»)	archivo/CRM	blue-collar, married, unknown, no, yes, no, unknown, may, unknown
y (respuesta del cliente)	binario 0/1	0 = no suscribe, 1 = suscribe	ambiente (`contactar()`), solo después de actuar	0
frac, tandas, fallo, seed	float / entero	frac ∈ (0, 1]; tandas ≥ 1; fallo ∈ [0, 1]; seed entero	usuario (CLI o Streamlit)	0.20, 5, 0.10, 42

La variable `duration` del archivo original se descarta en `preparar()` y nunca llega al agente.

9.4 Salidas y acciones

Acción	Método	Efecto en el entorno
Puntuar y ordenar el lote	`decidir()`	Calcula la probabilidad de compra de cada cliente; no cambia el entorno.
Contactar a la lista priorizada	`actuar()` → `contactar()`	Los clientes del 20 % superior reciben la oferta; el ambiente revela la respuesta de los alcanzados (≈ 90 %).
Exportar la lista	`actuar()`	Escribe `listas/lista_ciclo_N.csv` con índice del cliente y probabilidad.
Criticar la predicción	`criticar()`	Calcula Brier y sesgo de probabilidad y los registra.
Reentrenar	`aprender()`	Añade los alcanzados al conjunto de entrenamiento y ajusta de nuevo el modelo.
Rechazar un lote inválido	`percibir()`	No decide, no contacta ni reentrena; registra el evento `rechazo` con el motivo.
Registrar el ciclo	`_log()`	Añade una línea JSON con marca de tiempo a `log_ejecucion.jsonl`.

9.5 Diagrama de secuencia UML

Los diagramas coinciden con el código: cada mensaje corresponde a una llamada de `main()`/`app.py` o de un método de `CampaignTargetAgent`. Los participantes son el actor externo (equipo comercial), la interfaz, el agente separado en percepción, decisión, acción y crítico/aprendizaje, el modelo, el ambiente simulado y los servicios externos (UCI y archivos de salida).

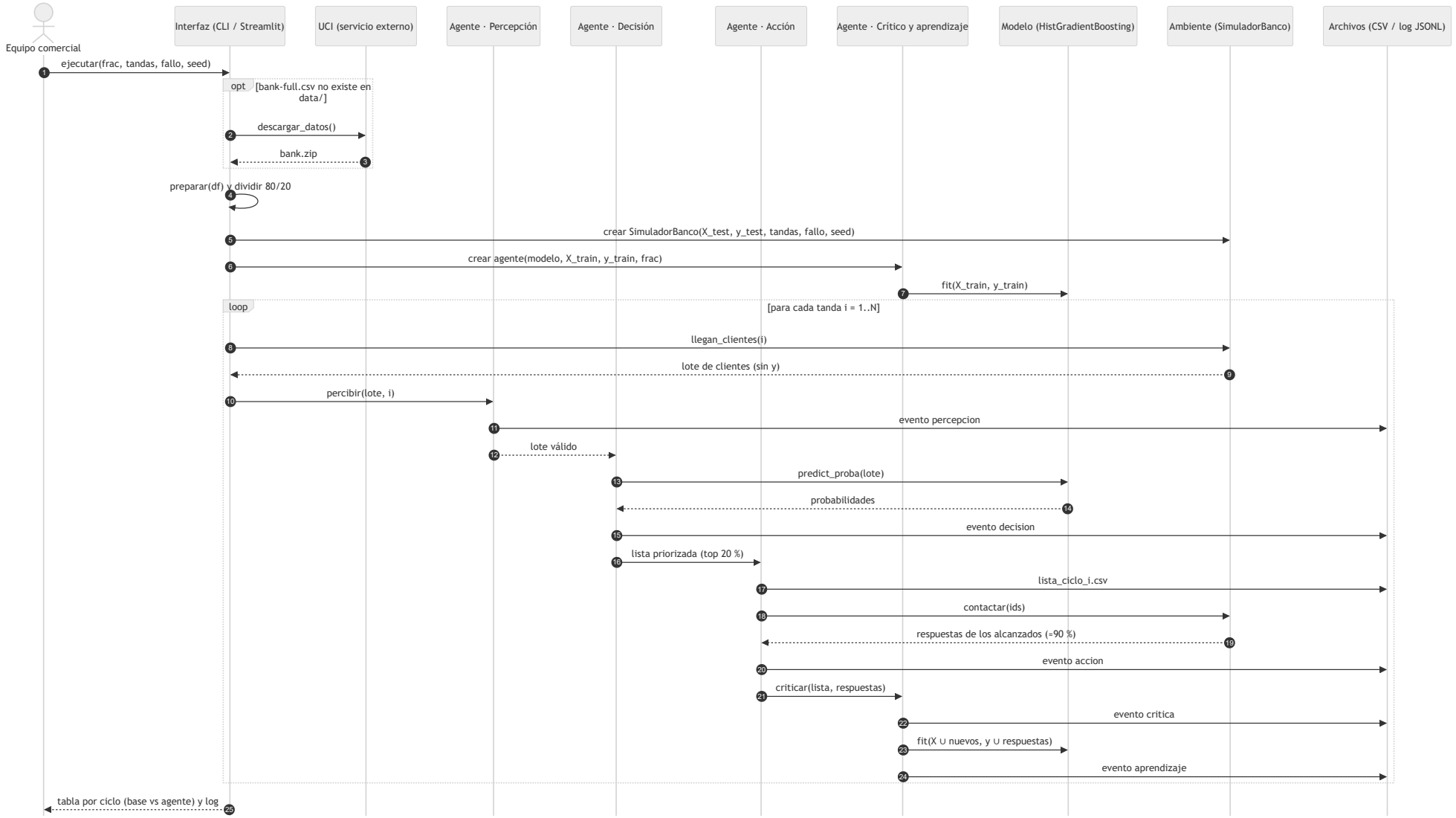


Figura 2. Escenario típico: ciclos de percepción, decisión, acción, crítica y aprendizaje (fragmentos *opt* y *loop*).

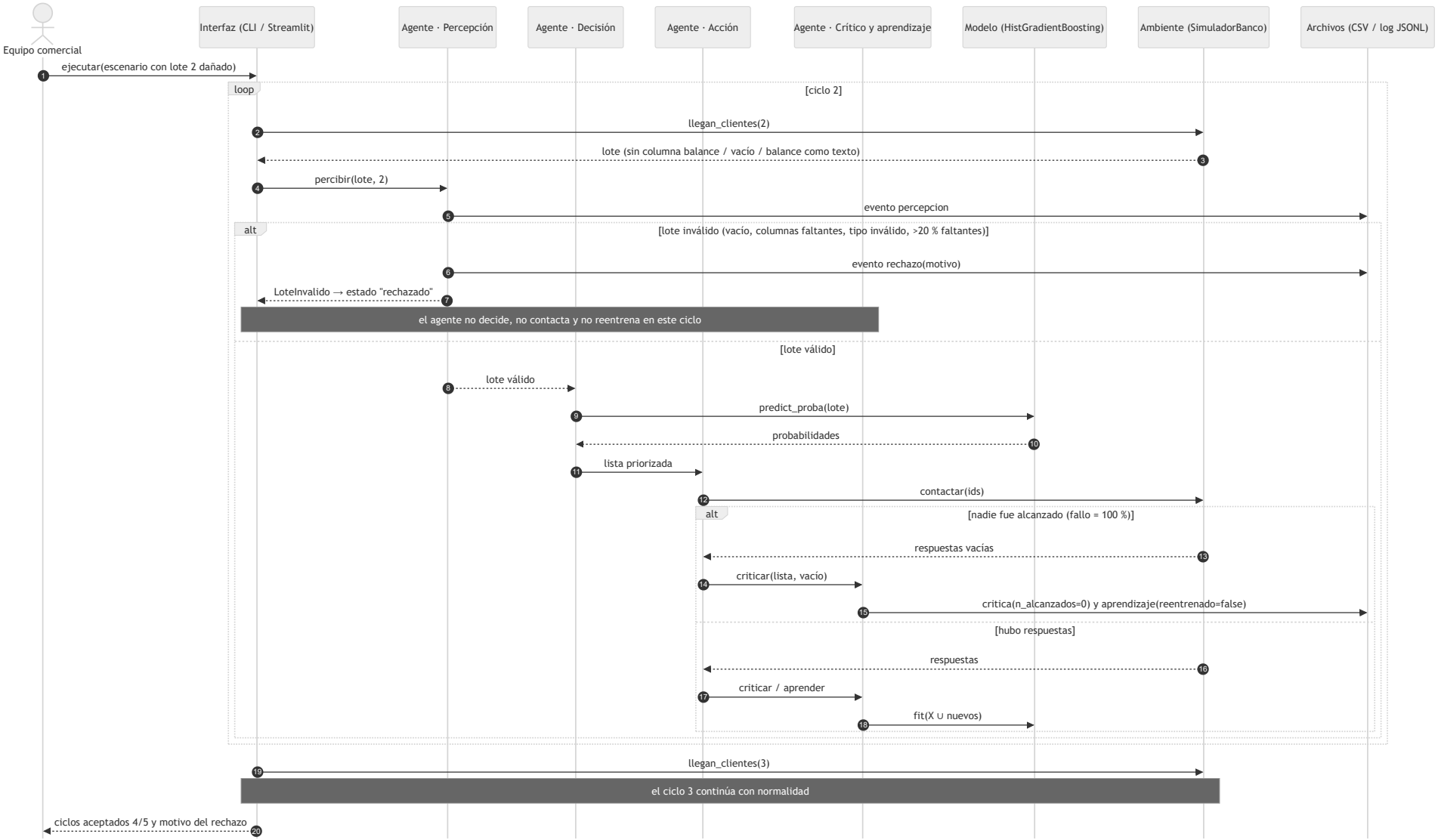


Figura 3. Escenario de excepción: lote inválido en el ciclo 2 y contactos sin respuesta (fragmentos *loop* y *alt* anidados).

9.6 Diferencias entre diseño e implementación

Diseño (Tarea 2 / Fase 1)	Implementación	Por qué
El actuador envía la lista a una plataforma de campañas o CRM.	`actuar()` escribe un CSV y llama a `SimuladorBanco.contactar()`.	No hay acceso a un CRM real; el simulador entrega las respuestas con 10 % de fallos (ambiente estocástico declarado).
Se excluye `duration`.	Igual.	— (se mantiene: evita fuga de información).
El crítico compara predicción y respuesta real.	Implementado como `criticar()` (Brier y sesgo); no modifica la decisión, solo informa.	En el código de la Fase 1 el crítico estaba en la descripción pero no en el código.
Medidas M1–M5.	Se añade M6 (robustez) y M5 pasa de 4 a 5 eventos por ciclo.	Exigencia de casos de falla esperada y consecuencia del crítico.
Se contacta el 20 % superior.	Parámetro `frac` (20 % por defecto), con mínimo de un cliente por lote.	Evitar una lista vacía en escenarios límite (1 %).
Validación de entradas limitada.	`percibir()` valida el esquema y rechaza lotes inválidos.	Hacer probables las fallas esperadas.
Sin generador de problemas.	Sin generador de problemas.	Se mantiene como trabajo futuro (exploración con contactos aleatorios).

9.7 Pruebas

Cada medida de la sección 7 se convirtió en al menos una prueba con resultado numérico. Los umbrales, la fórmula y los escenarios donde aplica cada medida se fijaron en `umbrales_fase3.json` antes de ejecutar (SHA-256 d62b12376c46b743…, fijado el 2026-10-18). Son 15 escenarios (5 normales, 4 límite y 6 de falla esperada) con 5 repeticiones cada uno (semillas 1–5) = 75 corridas; el ambiente es estocástico. La corrida duró 8 min en Python 3.12.10, scikit-learn 1.9.1.

Tabla de resultados

Medida de desempeño	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia
M1 Mejora de conversion al contactar el 20 % priorizado	conv_agente / conv_base - 1; conv_agente = compradores / clientes alcanzados de la lista; conv_base = tasa de compra del lote completo	≥ 0.2	25 corridas (N1,N2,N5,L3,L4)	media 2.128 (sd 0.053; min por escenario 2.097) escenarios que cumplen: 5/5	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl
M2 Calidad del ranking (AUC-ROC por lote)	media de roc_auc_score(y_real, prob) sobre los lotes aceptados con ambas clases	≥ 0.75	45 corridas (N1,N2,N3,N4,N5,L1,L2,L3,L4)	media 0.804 (sd 0.002; min por escenario 0.803) escenarios que cumplen: 9/9	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl
M3 Captura de compradores en el 20 % superior	media por lote de compradores_en_lista / compradores_en_lote	≥ 0.5	25 corridas (N1,N2,N5,L3,L4)	media 0.625 (sd 0.005; min por escenario 0.623) escenarios que cumplen: 5/5	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl

Medida de desempeño	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia
M4 Estabilidad de la mejora entre repeticiones	desviacion estandar de M1 entre las 5 repeticiones	≤ 0.1	25 corridas (N1,N2,N5,L3,L4)	sd media 0.050 (max 0.085) escenarios que cumplen: 5/5	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl
M5 Registro completo y ordenado	ciclos con la secuencia de eventos esperada y marcas de tiempo no decrecientes / ciclos procesados (ok: percepcion, decision, accion, critica, aprendizaje; rechazado: percepcion, rechazo)	≥ 1.0	75 corridas (N1,N2,N3,N4,N5,L1,L2,L3,L4,F1,F2,F3,F4,F5,F6)	media 1.000 (sd 0.000; min por escenario 1.000) escenarios que cumplen: 15/15	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl
M6 Robustez ante entradas invalidas y entorno adverso (medida anadida en la Fase 3)	ciclos terminados de forma controlada y correcta / ciclos totales; correcto = lote con dano inyectado rechazado y registrado, o lote sano procesado sin excepcion no manejada	≥ 1.0	75 corridas (N1,N2,N3,N4,N5,L1,L2,L3,L4,F1,F2,F3,F4,F5,F6)	media 1.000 (sd 0.000; min por escenario 1.000) escenarios que cumplen: 15/15	Sí	resultados_fase3/ (corridas.csv, resultados_por_escenario.csv) y logs_fase3/*.jsonl

Resultados por escenario (media de 5 repeticiones)

ID	Escenario	M1 mejora ± sd	M2 AUC	M3 captura	M5 log	M6 robustez	Ciclos aceptados	Cumple (✓/✗)
N1	Operacion de diseno: 20 %, 10 % de fallo, 5 tandas	2,097 ± 0,028	0,804	0,625	1,000	1,000	5.0	M1✓ M2✓ M3✓ M4✓ M5✓ M6✓
N2	Sin contactos fallidos	2,144 ± 0,022	0,804	0,628	1,000	1,000	5.0	M1✓ M2✓ M3✓ M4✓ M5✓ M6✓
N3	Contactar solo el 10 %	3,479 ± 0,030	0,803	0,447	1,000	1,000	5.0	M2✓ M5✓ M6✓
N4	Contactar el 30 %	1,390 ± 0,017	0,804	0,717	1,000	1,000	5.0	M2✓ M5✓ M6✓
N5	10 tandas (mas ciclos de aprendizaje)	2,135 ± 0,054	0,804	0,625	1,000	1,000	10.0	M1✓ M2✓ M3✓ M4✓ M5✓ M6✓
L1	Contactar solo el 1 %	5,542 ± 0,167	0,804	0,066	1,000	1,000	5.0	M2✓ M5✓ M6✓
L2	Contactar al 100 % (prueba nula, sin priorizacion)	-0,009 ± 0,012	0,805	1,000	1,000	1,000	5.0	M2✓ M5✓ M6✓ nula✓
L3	50 % de contactos fallidos	2,135 ± 0,085	0,803	0,625	1,000	1,000	5.0	M1✓ M2✓ M3✓ M4✓ M5✓ M6✓
L4	20 tandas de ~450 clientes	2,127 ± 0,062	0,804	0,623	1,000	1,000	20.0	M1✓ M2✓ M3✓ M4✓ M5✓ M6✓
F1	Entorno adverso: 90 % de contactos fallidos	1,955 ± 0,449	0,805	0,625	1,000	1,000	5.0	M5✓ M6✓
F2	Lote 2 sin la columna balance	2,102 ± 0,040	0,802	0,623	1,000	1,000	4.0	M5✓ M6✓
F3	Lote 2 con balance sin dato (todo NaN)	2,102 ± 0,040	0,802	0,623	1,000	1,000	4.0	M5✓ M6✓
F4	Lote 2 vacio	2,102 ± 0,040	0,802	0,623	1,000	1,000	4.0	M5✓ M6✓
F5	Nadie es alcanzado (100 % de fallo)	—	0,803	0,626	1,000	1,000	5.0	M5✓ M6✓
F6	Lote 2 con balance como texto	2,102 ± 0,040	0,802	0,623	1,000	1,000	4.0	M5✓ M6✓

M1, M3 y M4 solo aplican al punto de operación de diseño (frac = 0,20) en escenarios normales y límite; M2 aplica a todos los escenarios no marcados como falla; M5 y M6 aplican a todos. «nula» es la prueba nula de L2 (|M1| ≤ 0,05).

Corrida inicial en Colab y reformulación de las reglas

Antes de este documento, uno de los integrantes ejecutó una primera suite en Google Colab (12 escenarios × 5 repeticiones = 60 corridas) con el agente original, que no validaba entradas, y con los umbrales M1–M5 aplicados a *todos* los escenarios. Sus números de N1 coinciden con los de la corrida entregada (+213 %, AUC 0,804, captura 62,5 %), lo que corrobora ambos entornos. Esa corrida tuvo siete incumplimientos, que motivaron dos cambios: (1) el agente ahora valida entradas en `percibir()`; (2) se declaró en `umbrales_fase3.json` en qué escenarios aplica cada medida y se añadió M6. **Esta segunda reformulación se hizo después de ver la corrida inicial**, así que se muestran ambas lecturas: la columna «regla original» aplica los mismos umbrales a todos los escenarios, sin excepción.

Escenario	Colab, agente original (60 corridas)	Agente corregido, regla original (esta entrega)
N1 base	sin incumplimientos	sin incumplimientos
N2 sin fallos de contacto	sin incumplimientos	sin incumplimientos
N3 contactar 10 %	M3 (44 % de captura)	M3 (44,7 %)
N4 contactar 30 %	sin incumplimientos	sin incumplimientos
N5 10 tandas	no existía	sin incumplimientos
L1 contactar 1 %	M3 y M4	M3 y M4
L2 contactar 100 %	M1 (+0,4 %)	M1 (−0,9 %)
L3 50 % de fallos	sin incumplimientos	sin incumplimientos
L4 20 tandas	sin incumplimientos	sin incumplimientos
F1 90 % de fallos	M4	M4 (sd 0,449)
F2 sin columna balance	**M5 (85 %)**: el ciclo 2 fallaba con excepción	sin incumplimientos: lote rechazado y registrado
F3 balance sin dato (NaN)	sin incumplimientos (el boosting acepta NaN)	sin incumplimientos: ahora se rechaza por más de 20 % faltante
F4 lote vacío	**M5 (85 %)**	sin incumplimientos: lote rechazado y registrado
F5 nadie alcanzado	no existía	M1 y M4 indefinidas (sin respuestas)
F6 balance como texto	no existía	sin incumplimientos: rechazado

La lectura honesta: el diseño de las pruebas original ya detectaba el defecto real del agente (sin validación), y se corrigió; los demás incumplimientos con la regla original (M3 al 10 % y al 1 %, M1 al 100 %, M4 en 1 % y 90 % de fallos) no son defectos del agente sino umbrales fuera de su punto de operación o ruido de muestras pequeñas, y por eso se declararon no aplicables. Quien prefiera la regla estricta puede leer la columna derecha; los datos están en `resultados_fase3/resultados_regla_original.csv` (generado por `regla_original.py`).

Análisis de resultados

Qué se cumplió. Las seis medidas superan su umbral en todos los escenarios donde aplican: M1 con una mejora media de +213% (umbral +20 %; el peor escenario, +210%), M2 con AUC 0,804 (umbral 0,75), M3 con captura de 62,5% (umbral 50 %), M5 y M6 al 100 %. La prueba nula también pasa: al contactar al 100 % (L2) la mejora es -1%, es decir, la métrica no infla el resultado cuando no hay priorización. Los cuatro tipos de lote dañado (F2, F3, F4, F6) se rechazaron en el ciclo 2 con un motivo distinto registrado (`columnas_faltantes`, `datos_faltantes`, `lote_vacio`, `tipo_invalido`) y el agente procesó los ciclos 3–5.

Lo que los números no muestran tan bien.

1. **M4 queda al límite cuando el entorno se degrada.** La mayor desviación entre repeticiones entre los escenarios donde aplica fue 0,085 (L3, 50 % de contactos fallidos) frente a un umbral de 0,10, y en F1 (90 % de fallos), donde no aplicaba por el diseño previo de las pruebas, la desviación fue 0,449: M4 se habría incumplido. La causa probable es el tamaño de muestra: con 90 % de fallos solo se observan unas 36 respuestas por ciclo. Corrección propuesta: exigir un mínimo de contactos alcanzados por ciclo para evaluar M1/M4, o calcular M1 con intervalo de confianza en lugar de punto.
2. **La mejora con el entrenamiento completo no prueba que el agente «aprenda».** Con 36.168 clientes de entrenamiento, cada ciclo añade unas 330 respuestas (≈ 1 %): M1 en 10 tandas (N5, +214%) es indistinguible de 5 tandas (N1, +210%) y el AUC no cambia (0,804 frente a 0,804). Para comprobar la propiedad que define a un agente de aprendizaje se hizo un experimento complementario (`experimento_aprendizaje.py`, no pertenece a las medidas del diseño): el agente arranca con solo 300 clientes y se compara con una copia congelada que no reentrena, sobre los mismos lotes y contactos (20 tandas, 5 semillas).

Agente	Tramo	Mejora de conversión (media ± sd entre 5 semillas)	AUC-ROC por lote (media ± sd)
aprende	ciclos 1-5	+104% ± 27 pp	0,678 ± 0,039
aprende	ciclos 16-20	+166% ± 23 pp	0,748 ± 0,022
congelado	ciclos 1-5	+74% ± 16 pp	0,647 ± 0,031
congelado	ciclos 16-20	+87% ± 24 pp	0,666 ± 0,038

Con poca experiencia inicial, el agente que aprende pasa de +104% a +166% y de AUC 0,678 a 0,748, mientras que el congelado se mueve de +74% a +87% (AUC 0,647 a 0,666). Hay aprendizaje real, pero solo se nota cuando la base inicial es pequeña; la desviación entre semillas (16–27 pp) es comparable al salto en la mejora, por lo que la evidencia más clara es la del AUC. Con un modelo ya entrenado con la base completa el aporte del reentrenamiento por ciclo es marginal.

3. **La fracción del 20 % es una elección, no un óptimo.** Al contactar menos, la mejora sube y la captura baja: 10 % (N3) da +348% con 44,7% de captura; 30 % (N4) da +139% con 71,7%; 1 % (L1) da +554% pero captura 6,6%. Por eso M1 y M3 solo se exigieron en el punto de operación del diseño. Sin una función de utilidad con el costo de la llamada no se puede decir cuál fracción es mejor.
4. **Sin respuestas, M1 no está definida.** En F5 (100 % de contactos fallidos) nadie responde, M1 es indefinida (no hay conversión que medir). El agente lo gestionó bien (registró `critica` con `n_alcanzados = 0`, no reentrenó y completó los 5 ciclos: M6 = 100 %), pero la medida debe reformularse como «definida solo si hay clientes alcanzados».
5. **Alcance de la evidencia.** Las repeticiones solo varían el ambiente (orden de lotes y fallos de contacto); el modelo inicial y la partición 80/20 son fijos, por lo que la desviación estándar subestima la variabilidad real. Los cuatro escenarios de lote dañado comparten semillas y dan el mismo M1 (+210%): comprueban la validación, no aportan cuatro estimaciones independientes del desempeño.

Qué cambiaría el equipo: añadir contactos aleatorios (generador de problemas) para reducir el sesgo de retroalimentación, fijar un mínimo de respuestas por ciclo para M1/M4, y repetir las pruebas variando también la partición.

9.8 Limitaciones y trabajo futuro

- **Sesgo de retroalimentación.** El agente solo aprende de los clientes que contacta; los que descarta nunca generan datos. Trabajo futuro: reservar una fracción de contactos aleatorios (generador de problemas) y medir su efecto.
- **Backtest, no piloto.** Las pruebas usan datos históricos de 2008–2010 de una sola entidad portuguesa y un simulador; un piloto real puede dar una mejora menor. La conversión del simulador se revela con las etiquetas históricas, no con clientes vivos.
- **Equidad y privacidad.** Edad y estado civil entran al modelo sin auditoría de sesgo por grupo; falta evaluar el consentimiento y la normativa de datos antes de cualquier uso real.
- **Variación entre repeticiones acotada.** En las repeticiones solo varía el ambiente (orden de lotes y contactos fallidos); el modelo inicial y la partición 80/20 son fijos (semilla 42). La desviación estándar de M1 subestima la que habría con particiones distintas.
- **Fracción fija.** El 20 % no se optimiza frente al costo de la llamada; sería una función de utilidad con ingreso esperado por suscripción menos costo de contacto.
- **Interfaz.** La comunicación es tabular; falta explicar por cliente por qué quedó alto (por ejemplo, con importancia de variables o valores SHAP).

9.9 Enlaces

- Repositorio: https://github.com/WhiterBy1/campaign-target-agent
- Plataforma de despliegue (opción A, Streamlit Community Cloud): https://campaign-target-agent.streamlit.app/ — instrucciones y casos de prueba sugeridos en la pestaña «Cómo probar» de la aplicación.
- Logs de las pruebas: carpeta `logs_fase3/` y `resultados_fase3/` del repositorio.

10. Conclusiones

El diseño es viable y la implementación lo confirma. Usa un conjunto de datos público, modelos estándar y un costo de cómputo mínimo (el entrenamiento y las 75 corridas de prueba se ejecutan en minutos en un computador personal). En el punto de operación de diseño (20 % superior) el agente obtiene una mejora de conversión de cerca de +210 % frente a no priorizar, con AUC-ROC de 0,80 y captura de cerca del 63 % de los compradores; los resultados de cada medida, incluidas las que no se cumplieron, se discuten en la sección 9.7.

Retos para la siguiente fase. Primero, el sesgo de retroalimentación: habría que reservar una fracción de contactos aleatorios para explorar. Segundo, confirmar el backtest con un piloto real. Tercero, la equidad: auditoría de sesgo por edad y estado civil. Cuarto, la privacidad y el consentimiento del cliente. Quinto, la deriva de los datos, pues el conjunto procede de campañas de 2008 a 2010 de una sola entidad. Por último, optimizar la fracción de contacto frente al costo real de cada llamada.

Anexo A. Dataset Bank Marketing (UCI)

Este anexo se incorpora al documento base para que ambos viajen juntos. **Fuente:** Moro, Rita y Cortez (2014), UCI Machine Learning Repository, https://doi.org/10.24432/C5K306 (licencia CC BY 4.0); campañas de telemercadeo de un banco portugués (2008–2010). **Archivo usado:** `bank-full.csv`, 45.211 registros, 16 variables de entrada y el objetivo `y` (¿suscribió un depósito a plazo?).

- **Desbalance:** 88,3 % «no» y 11,7 % «sí»; por eso se usa `class_weight="balanced"` y la partición 80/20 es estratificada (36.168 entrenamiento, 9.043 prueba, 11,7 % de conversión en ambos).
- **Datos desconocidos:** `poutcome` es «unknown» en 81,7 % de los registros y `contact` en 28,8 %; se tratan como categoría propia.
- **Fuga de información:** `duration` (duración de la llamada) se conoce solo después de contactar al cliente; se excluye.
- **Preparación:** `pdays = -1` pasa a la bandera `nunca_contactado`; variables categóricas en one-hot; 42 columnas finales (ver tabla de entradas en 9.3).
- **Limitación:** datos de una sola entidad y época; la generalización debe confirmarse con un piloto.

Referencias

- Russell, S. J., & Norvig, P. Artificial Intelligence: A Modern Approach (Global Edition).
- IBM. «Components of AI agents». https://www.ibm.com/mx-es/think/topics/components-of-ai-agents
- Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. Decision Support Systems, 62, 22–31.
- Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5K306
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Gilboy, N., Tanabe, T., Travers, D., & Rosenau, A. M. (2020). Emergency Severity Index (ESI) v4. AHRQ. (Agente descartado en la Fase 1.)